Gambling Host Tracker

Architecture and Technology

A collection system for the mobile-wallet and bank account numbers that gambling sites publish for deposits, built so that an AML team can blocklist them and evidence the decision.

Document Architecture and technology
Status Current as built
Generated 23 August 2026
Source docs/architecture.md

Contents

1. Purpose and scope

Gambling sites operating in Bangladesh publish mobile-wallet and bank account numbers on their deposit pages so that players can send them money. Those numbers are the point at which gambling money enters the regulated banking system, and they are exactly what an AML team needs in order to blocklist the receiving accounts.

The numbers rotate two or three times a day. A screenshot taken this morning describes an account that may no longer be advertised this afternoon, and an account that has stopped being advertised was still collecting deposits an hour ago. A system that reports only the current value therefore answers the wrong question. This one records **every sighting**, keeps the page each sighting came from, and hashes that page so the record can be shown later to be the same bytes the number was read from.

What the system produces is a timeline: which account was advertised, on which site, at which time, with evidence attached.

Explicitly out of scope. The system never creates an account on a gambling site, never completes a payment, and never defeats a CAPTCHA. Two configured methods reach their payee by confirming a deposit, which raises a deposit *request* on the operator; no funds move and nothing is ever paid. Collected data includes account holder names and numbers and is to be handled under the organisation's existing PII and AML retention policy.

2. System overview

The system is a single Python application with four faces onto the same core:

Face	What it is for
Collector	Drives a real browser through a site's deposit flow and records what it finds
Portal	A read-only, server-rendered web interface over the collected data
CLI	Operational commands: run, recon, export, verify, status
Scheduler	Runs a collection on an interval without anyone at the keyboard

A typical working day is: the operator opens the portal, starts a collection from the Runs page, signs in once when the site shows its CAPTCHA, and the run completes by itself. The payees it brought back are then filtered, exported to CSV for the blocklist feed, or printed as a PDF report for a case file.

At a glance

Language	Python 3.12 (targets 3.11+)
Source	~6,400 lines of Python, ~1,850 lines of HTML templates
Tests	217 test functions, 250 assertions-level cases, entirely offline
Database	SQLite locally, PostgreSQL in production, no code change between them
Schema	11 tables, 4 Alembic migrations
Targets configured	2 (`1xbet-bd`, `melbet-bd`)

3. Architecture

3.1 Layers

The application is layered so that each stage can be tested against saved input without the stage before it. The dependency direction is strictly downward.

Layer	Modules	Responsibility
Interface	`cli.py`, `api/routes`, `api/templates`	Commands and pages. No domain logic.
Orchestration	`pipeline/run.py`, `api/jobs.py`, `api/schedule.py`	Sequencing one run, and deciding when a run may start
Acquisition	`fetchers/`, `auth_login.py`, `browserlaunch.py`	Getting the bytes: sessions, browsers, click flows
Interpretation	`extractors/`, `normalize/`	Turning a page into validated, canonical account records
Persistence	`models.py`, `pipeline/dedup.py`, `pipeline/changeset.py`, `pipeline/evidence.py`	Storing sightings, de-duplicating identities, keeping evidence
Configuration	`sources.py`, `sources/*.yaml`, `config.py`, `credentials.py`	What to collect and how, held outside the code

3.2 Why per-site behaviour lives in configuration

Every site-specific fact — the URL, the iframe the payment panel lives in, the selector that holds a Nagad wallet, the exact label of an option in a bank dropdown — is declared in `sources/<slug>.yaml` and never in code.

This is a deliberate structural choice with three consequences:

- **A site redesign is a configuration diff.** When an operator moves a button, the fix is one YAML line, reviewed and merged like any other change.
- **Git history becomes the audit trail.** For any date, the repository can answer "what was the collector looking for on that day", which matters when a stored piece of evidence is questioned months later.
- **Adding a site adds no code.** A new target is a new YAML file and two environment variables.

3.3 Request path of one collection

```
Portal "Start run"  or  scheduler tick  or  ght run
        |
        v
  machine-wide lock (data/run.lock)  --- held? -> refuse, name the holder
        |
        v
  detached subprocess: ght run --site <slug> --progress
        |                                  |
        |                                  +--> marker-prefixed JSON lines on stdout
        |                                       read by the portal, rendered as a
        |                                       three-step live checklist
        v
  sign in  ->  walk probes in one loaded panel  ->  store evidence  ->  extract
        ->  de-duplicate  ->  compute changes  ->  refresh active flags
```

4. Technology stack

Concern	Choice	Why this one
ORM and schema	SQLAlchemy 2.0 + Alembic	Typed declarative models; portable column types keep SQLite and PostgreSQL on one schema
Validation	Pydantic 2	Site configuration is validated on load, so a malformed YAML fails at startup with a named field rather than mid-run
HTML parsing	selectolax 0.4	C-backed; the collector parses the same large page twice per probe
Browser automation	Playwright 1.62 (Chromium)	The deposit panels are JavaScript applications inside iframes; nothing less will reach them
HTTP	httpx	For sites that render server-side and need no browser
Web framework	FastAPI + Jinja2	Server-rendered pages, no client-side framework, no build step
CLI	Typer + Rich	Command surface with readable terminal tables
PDF	ReportLab	Report generation and this document
Settings	pydantic-settings	One typed settings object over environment and `.env`
Lint and test	Ruff, pytest	100-column line length; the suite runs fully offline

Optional dependencies are separated into extras — `browser`, `api`, `export`, `sched`, `pg` — so that a machine which only needs the extractor does not install a browser engine, and the test suite runs without one.

5. Data model

5.1 Tables

Table	Holds	Written
`sites`	A configured target	Once per config change

Table	Holds	Written
site_urls	Known URLs and mirrors for a site	On config change; last_ok_at per run
collection_runs	One fetch attempt and its outcome	Once per run
evidence	Stored page bytes, addressed by SHA-256	Once per artefact per run
accounts	The de-duplicated account behind the sightings	Created once, then counters move forward
observations	One sighting of one account in one run	Append-only, never updated
account_sites	Which sites advertised which account	Upserted per run
merchant_sightings	Payees identified by name only	Append-only
excluded_numbers	Published numbers that are not deposit accounts	Rarely
alerts	Machine-detected conditions raised during a run	Per occurrence
access_log	Who searched, exported, ran or read	Per action

5.2 The two ideas the schema is built on

Observations are append-only. Overwriting a row would destroy the history that is the product. An account is a de-duplicated identity keyed on `(channel, account_number, bank_key)`; a sighting is an immutable fact about one moment.

Absence decays rather than flips. An account stays `is_active` for 48 hours after its last sighting. A number that rotated out this morning was still collecting deposits today and belongs on the blocklist this afternoon.

5.3 Two decisions worth recording

`bank_key` is an empty string rather than `NULL` for mobile-wallet accounts. `NULL` never compares equal in SQL, so a nullable column in the unique constraint would have inserted the same wallet again on every run.

Column types are deliberately portable — `JSON` rather than `JSONB`, no array columns — so that the local SQLite database and a production PostgreSQL instance run the same schema with no code path between them.

5.4 Payees without a number

Some deposit methods hand off to the payment provider's own checkout, which names a business, asks the payer for *their* wallet number, and never shows a receiving account. These cannot be stored as accounts, because the key they would be stored under does not exist.

They are kept as `merchant_sightings`: append-only, because a name not shown today says nothing about whether it is still in use, and the names rotate per request. Everywhere the system counts, charts, lists, exports or details a payee, both kinds are included. A run that collected one merchant and nothing else must not report

that it found nothing.

6. The collection pipeline

One run against one site proceeds in six stages.

1. Sign in. Before anything is fetched, not after a failure. See section 8.

2. Walk the probes. A site is configured as a list of probes, one per payment method. Each probe has its own click flow and its own selectors, because the markup differs per method. Where the site allows it, every probe is walked inside a single loaded panel.

3. Store evidence. The bytes the server returned, plus a screenshot, written under the SHA-256 of their own content — *before* extraction, so that a page which later turns out to break the selectors can be re-processed from exactly what was seen.

4. Extract. Two independent paths over the same page. See section 7.

5. Persist. Each validated account becomes an observation; the account row behind it is created or has its counters moved forward; the site link is touched.

6. Reconcile. Compare against the site's previous successful run to determine what is new, what has reappeared, and what has disappeared, then recompute active flags.

6.1 Sharing one panel across probes

The embedded payment panel takes ten to thirteen seconds to start, and every probe needs the same one. Re-fetching it per probe was the bulk of a run: 492 seconds for five methods.

The collector now opens the panel once and walks every probe inside it, closing the open modal between them. Crucially, the reset is *proven* rather than assumed — the configuration names a selector that must stop matching before the next probe may click. A half-closed modal would swallow the next click and be reported as that probe's selector being broken. Where the reset cannot be proven, the panel is reloaded; sharing it can cost time but cannot mix one method's payee up with another's.

The same five methods now take 76 seconds.

6.2 Blame and completeness are separate questions

This is the single most consequential piece of judgement in the system.

What happened	Run status	Reasoning
The site switched a method off and said so	`ok`	Their decision. Nothing here is broken and nothing can be fixed. Named in the run's note.
A configured selector no longer matches	`partial`	Ours to repair.
The saved session had expired	`failed`	Named as an expiry, with what to do about it.
Nothing loaded	`failed`	The site or the network.

What happened	Run status	Reasoning
A challenge page or 403	`blocked`	Tracked apart from `failed`: it needs a different fix.

Both failure modes of getting this wrong are real. A green `ok` beside an empty result is how a broken collector goes unnoticed for a week. But a permanent `partial` that nobody can clear teaches the same operator to ignore the colour entirely.

Separately from blame, a run tracks whether it saw *everything* it was meant to. When any probe did not complete, the run is marked incomplete and **no account is concluded to have disappeared** — an account the run could not reach looks identical to one that was taken down, and inferring absence from a partial view produces false retirements.

7. Extraction

Every page is extracted twice, on purpose.

The selector path runs the configured blocks: a container, the element holding the value, optionally the element holding the payee's name, and the channel stated as fact rather than inferred. Precise, and it knows what it found.

The regex sweep ignores the configuration entirely and scans the rendered text for anything shaped like a Bangladeshi payment account, guessing the channel from whichever brand name sits nearest. Noisy on its own.

The gap between the two is the health signal:

- Selectors find numbers — normal run, high confidence.
- Selectors find nothing **but the sweep still does** — the site was redesigned and the configuration is stale. The run is marked `partial` and an alert is raised, instead of quietly reporting zero.
- Neither finds anything — a genuinely empty page.

7.1 Confidence

Tier	Value	Meaning
High	0.9	Matched a configured selector that names the channel
Medium	0.6	Matched a selector; channel inferred from nearby text
Low	0.3	Found only by the sweep

Anything below high is held out of an automatic blocklist feed and routed to review.

7.2 Normalisation

A number alone never tells you the channel: a bKash wallet and a Nagad wallet can sit on the same mobile number. The channel comes from context — the label beside the field, the section heading, the CSS class — matched most-specific-brand-first, in both English and Bengali.

Mobile numbers are canonicalised to `+8801XXXXXXXXX`. Bengali numerals are translated first, so one pattern handles both scripts. The pattern is bounded by lookarounds so that it cannot bite eleven digits out of a seventeen-digit bank account number.

Two domain details that cost real bugs:

- **Rocket publishes twelve digits** — the wallet's mobile number with a check digit appended. The MSISDN pattern refuses that by design, so the number was being dropped entirely. It is now read as a wallet *only* when the configured block says the channel is Rocket, and keyed on the mobile, which is the identity; the printed string is preserved verbatim on the observation.
- **A number claimed by a selector is off-limits to the sweep.** The sweep decides a channel by proximity, which is often the *next* block's heading; without this rule one bKash wallet was also stored as a phantom Nagad account.

Bank accounts have no fixed shape in Bangladesh — thirteen digits at one bank, seventeen at another, twenty at a third — so the digits carry almost no signal. What makes a digit run a bank account is the bank name, branch and holder name around it, or a configured block that vouches for it.

8. Sessions, sign-in, and the browser

Deposit pages sit behind a login, and the sites use bot protection. A run therefore tries the cheapest thing that can work and escalates only when it must.

1. Is the saved session still good? Checked headlessly, by loading the deposit page and waiting for *the payment application the collector actually reads* — not just the page around it. This distinction was a real defect: a site will serve the shell to an expired session and refuse only the embedded app, so the shallower check reported "signed in" and the first probe then reported being signed out.

2. Sign in unattended. With credentials in the environment, the run fills the form and submits in a hidden browser. If the site answers with a CAPTCHA or 2FA it stops there.

3. Ask the operator. A visible browser window opens, already filled in, leaving only the challenge and the button. The moment the success marker appears the session is captured and collection continues *within the same run*. If nobody finishes in five minutes, the run reports the expiry and stops.

Success is **proved before the session is saved**: the page collection actually needs is loaded and checked. Declaring success on a login-form heuristic is how a logged-out session gets saved and every subsequent probe fails.

8.1 Details that make this work in practice

- **The session carries its own browser identity.** Cloudflare binds its clearance cookie to the User-Agent that earned it. The saved state therefore wraps the Playwright storage state together with the User-Agent and the browser channel that produced it, and replay uses those.
- **Browser resolution falls through.** Playwright's bundled Chromium is preferred, but security software on locked-down Windows machines quarantines it without warning, so the launcher falls back to installed Edge and Chrome, and can be pointed at any Chromium build by path.
- **Credential-bearing URLs are redacted at capture.** Embedded payment panels take a short-lived token in the query string, and that URL is stored on the run and appears in exports. Nine parameter names are replaced with `REDACTED` before anything is persisted.
- **A refused connection is retried.** These hosts are reached over networks that drop them in bursts. One attempt per mirror turned a network hiccup into a failed run reporting that the site was down.

8.2 The flow engine

A probe's click path is declared as steps. A step performs exactly one action — click, select an option, or type a value — and may declare the frame it acts in and a selector to wait for afterwards.

Three capabilities exist because real sites required them:

- **Typing rather than assigning.** These forms enable their next button from the input's own key events; a value set directly onto the element leaves the button disabled and the following step reported as a stale selector.
- **Frame switching per step.** One aggregator is three documents deep: the method cell is in the deposit panel, the amount form is in the provider's iframe nested inside it, and confirming takes the whole tab to the payment provider's checkout.
- **Racing the expected panel against the site's own refusal.** Waiting only for the expected panel means a method the operator has switched off costs a full timeout and is then reported as our selector being broken.

9. Concurrency, scheduling and progress

One collection at a time, machine-wide. A collection drives a real browser against a live account; two at once race on the login session. The gate is a lock file holding a process id, so it holds across processes — a scheduled run and a hand-started CLI run cannot collide, and a second portal cannot start one either. A lock whose process is gone, or one older than any real collection, is cleared rather than left to block every future run.

Collections run as detached subprocesses. A run takes minutes and cannot occupy a web request.

Progress is streamed as text. The subprocess writes one marked JSON object per line to stdout; the portal parses those into a three-step checklist. The transport is deliberately dumb, because across a process boundary a line of text is the one channel that always works. Each update carries an explicit success flag: a run that ends `failed` is still a subprocess that ran to completion and exited zero, so the exit code cannot carry it.

The scheduler is deliberately small. One interval for one site, a thread that wakes and asks the same run manager the button asks. No queue, no catch-up, no second worker — the failure mode of a clever scheduler here is two browsers racing on one session. The floor is five minutes, the state survives a restart, and a tick that skips records *why* it skipped: a schedule that silently does nothing is indistinguishable from one that is not running.

10. The portal

Server-rendered pages over the collected data, bound to loopback.

Page	Answers
Overview	How much has been found, across which channels, and what arrived most recently
Payees	One list of both kinds of payee, searchable and filterable, with CSV and PDF export
Payee detail	The number, the holder, one screenshot of it on the site, and every sighting
Runs	Start a collection, set a schedule, and the paged history
Run detail	When a run went, how long it took, what it brought back, what it stored

Page	Answers
Components	The recurring interface elements and the reasoning behind them

Design decisions recorded in the code:

- The overview and the payees list **read the same query**, so the headline figure and the list behind it cannot drift apart.
- "Not applicable" and "unknown" are **different marks**. One means there is nothing to collect here; the other means there was something and we failed to collect it. A blank cell would let a reader mistake one for the other.
- There is **no alerts page**, on purpose. Whether collection is healthy is legible from the data itself.
- All timestamps are Bangladesh local time at +06:00, in one format, everywhere.

11. Evidence and data integrity

Whatever the server returned is written to disk verbatim and addressed by the SHA-256 of its own bytes. Two properties matter for a case file:

- the blob can be re-hashed at any time and shown to be the same content the number was extracted from, and
- identical pages across runs collapse onto one file, so months of three-a-day captures stay cheap while every run still gets its own evidence row.

Storage is sharded by the first two hex characters of the digest so no directory grows unbounded. `ght verify-evidence` re-hashes stored blobs against their recorded digests and names any that no longer match.

For each method a run reads, two artefacts are kept: the page the server returned, and a screenshot of it. The screenshot is the part a non-technical reviewer can actually read, and for a name-only payee it is the whole of the evidence.

12. Security, privacy and compliance posture

Control	Implementation
Credentials	Environment and a gitignored `.env` only. Never in YAML, the database, a log line or a run report. The credentials object redacts itself in its own `repr`, because it travels through exception handlers.
Tokens in URLs	Redacted at capture, before anything is stored or exported
Collected data in git	`data/` is gitignored in full — database, sessions, evidence, schedule, brand logos. Real collected account numbers never enter code, tests or comments.
Access accountability	Every search, export, run and read is written to `access_log` with actor, parameters and row count
CAPTCHA	Never defeated or worked around. A person clears it.
Portal exposure	No authentication; binds to `127.0.0.1` by design

Control	Implementation
Deposit side effects	Probes that raise a deposit request are flagged in configuration and named in the portal before a run

Before production. The portal must sit behind an authenticating reverse proxy. It must not be bound to `0.0.0.0` without one.

13. Testing

250 test cases across 14 modules, running entirely offline in about twelve seconds.

Area	What is covered
Flow engine	Click paths, dropdowns, frame switching, typed inputs, reset proof, unavailable-method races
Login	Session detection, escalation order, credential handling, session persistence
Portal templates	Every page and its empty states, rendered with hand-built contexts
Pipeline	Fetch, evidence, de-duplication and changeset logic
Extraction	Both paths, the merge rule, confidence, holder-name cleaning
Normalisation	Mobile numbers in both scripts, channels, bank accounts
Concurrency	Lock acquisition, stale-lock recovery, scheduling

Two choices keep the suite honest. The pipeline tests **serve saved fixtures over a local HTTP server**, so the fetcher, evidence store and de-duplication run the same way they do in production rather than against mocks. And the template tests render the real templates, because a broken template is a 500 on the one page an analyst is looking at, and no other test would catch it.

14. Deployment and operations

This needs a desktop. The assisted sign-in window must appear on a screen somebody is looking at. A headless server cannot show it, so a session refreshed there has to be copied in from a machine that can.

Local operation is a single double-click: `scripts\Start.bat` creates the virtual environment on first use, installs dependencies, downloads the browser, starts the portal and opens it.

For production, point `DATABASE_URL` at PostgreSQL — the models avoid Postgres-only column types precisely so that this needs no code change — and place the portal behind an authenticating proxy.

Commands

Command	Purpose
`ght run --site <slug>`	Collect from one site
`ght run --site <slug> --dry-run`	Extract and print, write nothing
`ght recon --site <slug>`	List the methods a site offers now, with their ids and dropdown labels

Command	Purpose
`ght recon --site <slug> --open <id>`	Describe one method's panel, element by element
`ght accounts` / `ght status`	What has been collected; how recent runs ended
`ght export --out <file>`	CSV for the AML team, recorded in the access log
`ght verify-evidence`	Re-hash stored blobs against their recorded digests
`ght serve`	Start the portal

15. Known limitations

- **No authentication on the portal.** Loopback-bound; requires a proxy before exposure.
- **Alert delivery is not implemented.** Alerts are detected and stored; the Teams, email and Telegram senders are not built.
- **Bengali names in PDF exports** need a Unicode font present on the machine. Without one the report prints ? rather than wrong glyphs. Untested against real Bengali data.
- **Assisted sign-in needs a person**, and on the sites configured today the CAPTCHA appears on essentially every sign-in.
- **Network reachability varies.** One target is intermittently refused at the TLS layer by local filtering; retries absorb this, at the cost of run duration.

Appendix A — Configured targets

Slug	Site	Methods collected	Notes
`1xbet-bd`	1xBet Bangladesh	5 probes across bKash, Upay, bank transfer and a name-only Nagad merchant	The site switches two or three methods off at any time
`melbet-bd`	Melbet Bangladesh	14 probes: CellFin, Nagad, Rocket, uPay, Nagad Free, Trust Axiata Pay, Rocket Free, iPay, Nexus Pay, four banks behind the Bank Transfer dropdown, and a name-only Nagad merchant	Same platform as 1xBet under different method ids

Both sites run the same underlying betting platform, which is why the login form, the embedded panel and the modal markup are shared. Every selector was nevertheless read off each site's own pages: the same engine is a reason to look, not a reason to assume.

Appendix B — Channels

`bkash`, `nagad`, `rocket`, `upay`, `tap`, `mcash`, `cellfin`, `ipay`, `bank_transfer`.

A channel is recorded as fact when a configured block names it, and as inference when it was read from the text around a number. The two are distinguished by confidence, never merged.

Appendix C — Glossary

Term	Meaning
Probe	One payment method's click path and selectors within a site configuration
Payee	Anything receiving deposits: a numbered account or a name-only merchant
Sighting	One observation of one payee in one run; never updated
Channel	The payment scheme a number belongs to
Evidence	Stored page bytes and screenshot, addressed by SHA-256
Reset	The proven-closed step that lets consecutive probes share one loaded panel
Creates order	A probe that reaches its payee by raising an unpaid deposit request